

Introduction

A **Map** is a collection that associates a key with a value. The mapping of keys to values can be accomplished using different underlying data structures. In this assignment, you will be implementing a **Map** using an underlying **Tree** data structure.

A tree can be defined recursively as a collection of nodes (starting at a root node), where each node is an object consisting of a value, together with references to nodes (the “children”), with the constraints that no reference is duplicated, and none points to root. A **Binary Search Tree (BST)** is a specific type of tree where each node can only have up to two children, and all of the data values in the left subtree of any node are always less than the node value and all of the data values in the right subtree of any node are always greater than the node value.

In this assignment you will be implementing a Binary Search Tree that implements the CSMaP interface.

Since the key-value pairs stored in the map could be of any type, we will be using Generics to implement all our classes. For example, we could have ("Andrew" -> 1) as a key-value pair (which is of type <String, Integer>) and we could also have (432 -> 34) as a key-value pair (which is of type <Integer, Integer>). Hence, it is important that we provide a way to reuse the same code with different inputs. Generics enable types (classes and interfaces) to be parameters when defining classes, interfaces and methods. Much like the more familiar formal parameters used in method declarations, type parameters provide a way for you to re-use the same code with different inputs. The difference is that the inputs to formal parameters are values, while the inputs to type parameters are types.

Note: You are NOT allowed to use any built-in Java data structures for any part of this assignment.

Tasks

Starter Code

You are being provided with starter code which is an interface representing a CSMaP. You can find it in the course public folder ([public/6P](#)). It contains the signatures of the methods that need to be implemented.

CSTreeMap

You can choose to implement CSTreeMap using iteration or recursion, as long as it is a binary search tree. You may add private methods as needed. You are required to implement the following methods for the CSTreeMap class:

- `public V get(K key)` - returns the value associated with the key or null if this map does not contain a mapping to this key.
- `public boolean insert(K key, V value)` - If the key does not already exist in the Map, insert the key, value pair and return true, otherwise return false.
- `public void clear()` - resets to the default empty state.
- `public boolean containsKey(K key)` - returns true if the map contains the key, otherwise returns false.
- `public boolean isEmpty()` - returns true if the map contains no key,value pairs, otherwise returns false.

- `public boolean remove(K key)` - If the key is present in the map, remove it and return true, otherwise return false.

Interactive Test Driver

Provides a user interface for creating and manipulating a `CSTreeMap` of String to Integer. Includes a command for each of the methods above. **It should exit when a user enters the command x.**

Command	Method
i	<code>insert(K key, V value)</code>
c	<code>containsKey(K key)</code>
C	<code>clear()</code>
g	<code>get(K key)</code>
s	<code>size()</code>
e	<code>isEmpty()</code>
r	<code>remove(K key)</code>

Sample Run

```

i zebra 100
(zebra->100)
-----
i kangaroo 50
(kangaroo->50) <--> (zebra->100)
-----
i apple 42
(apple->42) <--> (kangaroo->50) <-->
(zebra->100)
-----
i antelope 777
(antelope->777) <--> (apple->42) <-->
(kangaroo->50) <--> (zebra->100)
-----
i apple 100
(antelope->777) <--> (apple->42) <-->
(kangaroo->50) <--> (zebra->100)
-----
c apple
true
-----
c dog
false
-----
e
false
-----
s
4
-----
g kangaroo
50
-----
g zebra
100
-----
g cat
null
-----
r zebra
(antelope->777) <--> (apple->42) <-->
(kangaroo->50)
-----
r antelope
(apple->42) <--> (kangaroo->50)
-----
C
-----
e
true
-----
s
0
-----
x

```